

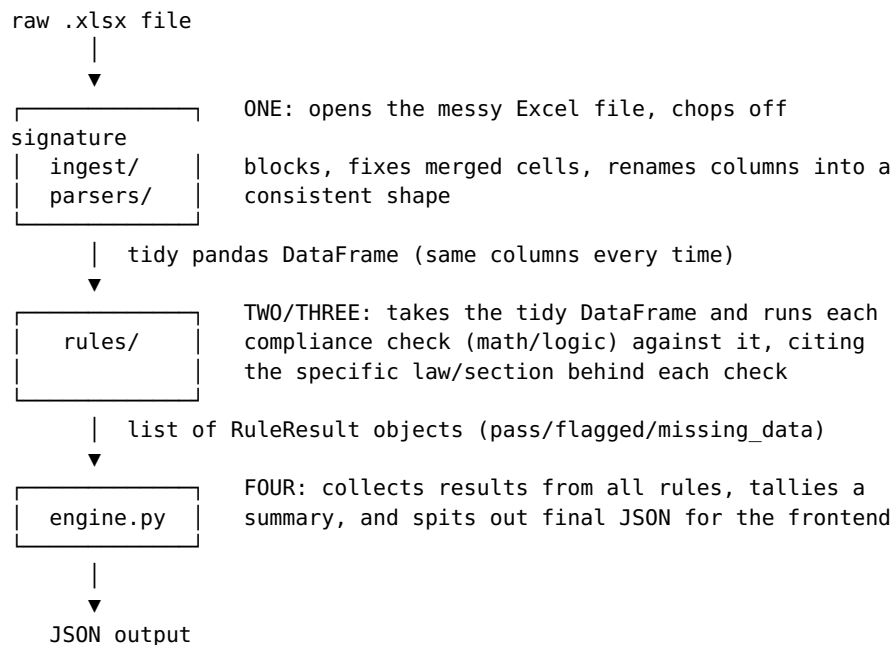
python_engine README

python_engine — README

This is SugboKlaro's compliance rule engine. It takes raw DILG Excel reports and turns them into Pass / Flagged / Missing Data results with legal citations attached, as clean JSON the frontend/backend can consume.

The four-stage flow

Think of it like an assembly line — each stage only cares about the stage before it, nothing more:



If you're adding a new document type (say, Pipeline A's budget reports), you're really just adding a new parser + a new set of rule classes and plugging them into `engine.py`'s dispatch table. Nothing else needs to change.

Folder structure

```
python_engine/
├─ config.py           # ALL file paths live here – see
                        "Paths" below
├─ conftest.py         # makes pytest work no matter what
                        folder you run it from
├─ engine.py           # orchestrator – ties parser + rules
                        together, outputs JSON
├─ requirements.txt
├─ ingest/
│   └─ schema.py       # defines what columns a tidy
                        DataFrame must have
├─ parsers/
│   └─ procurement.py  # Pipeline B: BRCWGS parser (CW, GS,
```

```

CS sub-forms)
├─ rules/
│   └─ base.py                # shared Rule / RuleResult /
RuleStatus classes
│   └─ procurement_rules.py    # Pipeline B: ABC ceiling, zero-
variance, vendor concentration
└─ tests/
    ├── test_procurement_parser.py
    └─ test_procurement_rules.py

```

Stage 1 — ingest/parsers/procurement.py

DILG's Excel files are messy: merged cells, inconsistent headers, footer signature blocks, multiple sub-forms per file. This file's only job is to turn that mess into a predictable table.

Current coverage: BRCWGS (Bid Results — Civil Works, Goods & Services, Consulting Services). Three sub-functions, one per Excel tab: - `parse_brcwgs_cw(filepath)` — Civil Works tab - `parse_brcwgs_gs(filepath)` — Goods & Services tab - `parse_brcwgs_cs(filepath)` — Consulting Services tab - `parse_brcwgs(filepath)` — combines all three into one DataFrame (this is the one engine.py actually calls)

Every parser returns a DataFrame matching `PROCUREMENT_SCHEMA` (defined in `ingest/schema.py`):

```

doc_type, reference_no, project_name, abc, winning_bidder,
bidder_address, bid_amount, bidding_date

```

Important design detail: if an LGU had zero bidding activity that quarter, DILG's own form says "NONE FOR THE PERIOD" instead of leaving it blank. Rather than treating that as an empty/missing form, the parser returns a real placeholder row (`reference_no = "NONE"`) — this correctly represents "submitted and compliant, just nothing to report," which is different from "form is missing." Every rule knows to skip these rows.

Consulting Services (BRCWGS_CS) never has an abc value — RA 9184's ABC ceiling doesn't apply to consulting contracts the same way. This is intentional, not missing data. Any new rule involving abc needs to explicitly exclude `doc_type == "BRCWGS_CS"` or it'll falsely flag every consulting row.

Stage 2 — rules/

`rules/base.py` defines the shared building blocks every pipeline uses:

- **Rule** — base class. Every rule is a small class with an `id` (e.g. "PROC-001"), a `legal_basis` dict (law/section/title), and an `evaluate(df)` method that returns a list of results.
- **RuleResult** — one finding: which rule fired, what status, a human-readable message, the legal citation, and which row it's about (`row_ref`).
- **RuleStatus** — enum: `PASS`, `FLAGGED`, `MISSING_DATA`.

`rules/procurement_rules.py` has the three checks currently implemented for Pipeline B:

Rule	ID	What it checks	Legal basis
		Bid amount doesn't	

ABCCeilingRule	PROC-001	exceed the Approved Budget for Contract	RA 9184 Sec. 31
ZeroVarianceBiddingRule	PROC-002	Flags when the winning bid lands within 1% of ABC (pattern associated with reduced competition — not an accusation)	RA 9184 Sec. 3
VendorConcentrationRule	PROC-003	Flags a vendor winning 3+ contracts in one quarter	RA 9184 Sec. 3

Tone rule: none of these should ever sound accusatory. ZeroVarianceBiddingRule and VendorConcentrationRule aren't named violations in the law — they're statistical patterns worth a human's attention, and the message wording reflects that ("an indicator commonly associated with...", not "this LGU violated...").

Stage 3 — engine.py

The dispatch table. This is the only file that needs to know about every pipeline that exists:

```
RULESETS = {
    "BRCWGS": (parse_brcwgs, PROCUREMENT_RULES),
    # Adding Pipeline A: "ABR": (parse_abr, BUDGET_RULES),
    # Adding Pipeline C: "UCA": (parse_uca, UCA_RULES),
}
```

run_check("BRCWGS", filepath) runs the full parse → evaluate → summarize flow and returns:

```
{
  "doc_type": "BRCWGS",
  "row_count": 26,
  "results": [ { "rule_id": "PROC-001", "status": "pass", "message":
"...", "legal_basis": {...}, "row_ref": "...", ... }, ... ],
  "summary": { "pass": 21, "flagged": 8, "missing_data": 3 }
}
```

This is the JSON contract the TS backend/frontend consumes.

Paths — config.py

Never hardcode a file path anywhere else in this project. Every path goes through config.py:

```
from config import BRCWGS_FILE
```

This anchors paths to the script's own location on disk rather than "whatever folder your terminal happens to be sitting in," which is what caused most of our early debugging headaches. If you add a new data file, add a new constant here — don't hardcode the string elsewhere.

conftest.py does the equivalent job for pytest, so tests work whether you run them from python_engine/ or the repo root.

Running things

```
cd python_engine
pip install -r requirements.txt

# Run the parser standalone (prints JSON of parsed rows)
python ingest/parsers/procurement.py

# Run the full engine (parser + rules + summary)
python engine.py

# Run all tests
python -m pytest tests/ -v
```

Known-good baseline (Cebu City Q1 2026 BRCWGS): **26 total rows, summary {"pass": 21, "flagged": 8, "missing_data": 3}**. This exact number is locked in as a regression test (test_full_summary_matches_known_good_run) — if it ever changes unexpectedly after an edit, something broke.

How to add a new document type (for whoever picks up Pipeline A or C)

1. Add a parser function in ingest/parsers/ (new file per pipeline, e.g. budget.py) that returns a tidy DataFrame with a consistent schema (define it in ingest/schema.py, same pattern as PROCUREMENT_SCHEMA).
2. Add a new rules/*.py file with rule classes subclassing Rule, each citing the relevant law.
3. Register both in engine.py's RULESETS dict.
4. Add a path constant to config.py for the new raw data file.
5. Write tests mirroring tests/test_procurement_*.py — at minimum, a “known good run” regression test locking in the real summary numbers once you’ve verified them against actual data.

Nothing in engine.py's run_check() function itself needs to change — it's already doc-type-agnostic.